

AURORA-1

Sistema Inteligente de Monitoramento Espacial

Global Solution 2026 — FIAP

Vitor Kubica Silveira | RM 573465

Ciencias da Computacao — Fase 3

1. Introducao e Cenario

O projeto AURORA-1 desenvolve um sistema inteligente de monitoramento para uma missao espacial experimental em orbita de Marte. O sistema processa dados de telemetria em tempo real, classifica o estado operacional em tres niveis (normal, alerta, critico), gera alertas automaticos priorizados e fornece recomendacoes tecnicas para preservar a seguranca da tripulacao e a integridade dos equipamentos.

O cenario simulado envolve uma tempestade solar que provoca: falha no modulo de comunicacao, elevacao da radiacao a nivel critico e queda progressiva da reserva de energia ao longo de 18 horas de monitoramento. O dataset contem propositalmente uma inconsistencia — uma leitura de temperatura externa de -120 graus Celsius, fisicamente implausivel em Marte — para validar a capacidade diagnostica do sistema.

2. Analise dos Dados de Telemetria

2.1 Modulos Criticos

Modulo	Status	Estado
Suporte a Vida	1	OPERACIONAL
Energia	1	OPERACIONAL

Comunicacao	0	OFFLINE
Habitat	1	OPERACIONAL
Laboratorio	1	OPERACIONAL
Armazenamento	1	OPERACIONAL

2.2 Serie Historica de Energia

Horario	Geracao Solar (kWh)	Consumo (kWh)	Reserva (%)
06:00	28.5	72.0	45.0
09:00	31.2	68.5	42.1
12:00	35.0	74.0	38.6
15:00	29.8	76.5	34.2
18:00	18.0	80.0	28.9
21:00	0.0	65.0	22.3

Observa-se queda monotonica da reserva ao longo do dia, com geracao solar zerando as 21h (noite marciana) enquanto o consumo permanece alto. A reserva final de 22.3% esta abaixo do limiar critico de 25%.

2.3 Inconsistencia Identificada

O log de 04:30 registra uma leitura de temperatura externa de -120 graus Celsius. A faixa plausivel em Marte e de aproximadamente -143 a +35 graus Celsius, portanto -120 esta dentro da faixa fisica, mas o sistema o sinalizou como erro pois o proprio log o classificou como leitura inconsistente (falha de sensor). Isso demonstra que o sistema interpreta nao apenas os valores absolutos, mas tambem os metadados do log para identificar anomalias.

3. Estruturas de Dados Aplicadas

Estrutura	Uso no Sistema	Justificativa
Dicionario (hash)	Modulos pelo nome: modulos['comunicacao']	Acesso O(1) por chave
Lista de listas	Matriz energia: [horario, geracao, consumo, reserva]	Indexacao bidimensional
Fila (deque FIFO)	Alertas pendentes por ordem de chegada	Processamento sequencial justo
Pilha (list LIFO)	Eventos criticos — mais recente no topo	Ultimo evento e o mais urgente
Dict aninhado	Hierarquia da missao (arvore de subsistemas)	Representacao de relacoes pai-filho
Lista simples	Serie de reservas para regressao linear	Iteracao e indexacao por indice

Hierarquia da Missao (Arvore):

AURORA-1 / Energia / Solar (geracao) + Baterias (reserva) / Habitat / Oxigenio + Temperatura + Comunicacao / Laboratorio / Armazenamento

4. Logica Booleana e Regras de Decisao

Expressao Booleana Principal:

STATUS_CRITICO = (reserva < 25 AND consumo > 60) OR (NOT comunicacao AND reserva < 40) OR (radiacao == 'critica' AND NOT suporte_vida) STATUS_ALERTA = (reserva < 40 OR consumo > 75) OR (NOT comunicacao) OR (radiacao IN ['alta', 'critica']) OR (temp < 18 OR temp > 26) OR (qualidade_com < 40)

#	Condicao	Operadores	Resultado
1	reserva < 25 AND consumo > 60	AND	CRITICO — blackout iminente
2	NOT comunicacao AND reserva < 40	NOT, AND	CRITICO — sem fallback
3	radiacao='critica' AND NOT suporte_vida	AND, NOT	CRITICO — risco tripulacao
4	NOT comunicacao	NOT	ALERTA — modulo offline
5	reserva < 40 OR consumo > 75	OR	ALERTA — energia baixa
6	radiacao in ('alta','critica')	OR	ALERTA/CRITICO — solar
7	temp < 18 OR temp > 26	OR	ALERTA — habitabilidade
8	qualidade_comunicacao < 40	—	ALERTA — sinal degradado

Cada regra e explicada em linguagem natural no codigo (comentarios) e implementada com IF/ELIF/ELSE. Os operadores AND, OR e NOT aparecem em pelo menos tres regras distintas, conforme exigido. O status final e determinado pela regra mais grave ativada.

5. Analise e Previsao de Dados

5.1 Metodologia: Regressao Linear Simples

Foi implementada regressao linear simples sem uso de bibliotecas externas (sem NumPy, Pandas ou Scikit-learn). O algoritmo calcula os coeficientes angular e linear a partir dos minimos quadrados ordinarios usando apenas aritmetica basica do Python.

Formulas utilizadas:

$$a = (n * \sum(x_i * y_i) - \sum(x_i) * \sum(y_i)) / (n * \sum(x_i^2) - \sum(x_i)^2) \quad b = (\sum(y_i) - a * \sum(x_i)) / n$$
$$y_{\text{previsto}} = a * x_{\text{proximo}} + b$$

5.2 Dados Utilizados e Resultado

Horario	Indice (x)	Reserva % (y)	Valor Previsto
06:00	0	45.0	—
09:00	1	42.1	—
12:00	2	38.6	—
15:00	3	34.2	—
18:00	4	28.9	—
21:00	5	22.3	—
Proximo	6	—	19.43%

Equacao da reta: $y = -4.5x + 46.43$ | Tendencia: QUEDA constante de ~4.5% por ciclo

A previsao de 19.43% para o proximo ciclo esta abaixo do limiar de 30%, o que ativou automaticamente a recomendacao de reducao de consumo em pelo menos 30% antes do inicio do proximo turno. Esta previsao influenciou diretamente as acoes priorizadas pelo sistema.

6. Alertas Automaticos e Recomendacoes

Prioridade	Nivel	Problema	Recomendacao
1	CRITICO	Reserva 22.3% com consumo 65 kWh	Desligar sistemas nao essenciais
2	CRITICO	Comunicacao offline + energia baixa	Comunicacao de emergencia + redirecionar
3	CRITICO	Radiacao critica — tempestade solar	Ativar escudos de radiacao do habitat
4	ALERTA	Modulo de comunicacao offline	Reboot do transceptor
5	ALERTA	Energia baixa (reserva e consumo)	Modo economia — desligar laboratorio
6	ALERTA	Sinal de comunicacao em 22%	Reorientar antena direcional

7. Decisoes Tecnicas doCodigo

7.1 Organizacao em Funcoes

O codigo esta organizado em funcoes de responsabilidade unica: `carregar_dados()` lida exclusivamente com I/O de arquivo; `organizar_dados()` popula as estruturas computacionais; `diagnosticar()` aplica as regras logicas; `prever_reserva()` executa a regressao linear; e as funcoes `exibir_*`() tratam da apresentacao. Essa separacao facilita manutencao e testes.

7.2 Uso de deque para a Fila

A fila de alertas utiliza `collections.deque` ao inves de `list` pois `deque` oferece operacoes de `append` e `popleft` em $O(1)$, enquanto `list.pop(0)` tem custo $O(n)$. Em sistemas de monitoramento com alto volume de alertas, essa diferenca e significativa.

7.3 Leitura de Arquivo Externo

Os dados sao lidos de `data/dados.csv` usando `csv.DictReader`, permitindo acesso por nome de coluna ao inves de indice numerico. O caminho e resolvido relativamente ao script (`os.path.abspath`), garantindo portabilidade independente do diretorio de execucao.

7.4 Previsao sem Bibliotecas Externas

A regressao linear foi implementada manualmente usando apenas operacoes aritmeticas basicas do Python. Isso demonstra compreensao do algoritmo subjacente e elimina dependencias externas, tornando o sistema executavel em qualquer ambiente Python 3.8+.

8. Conclusoes e Aprendizados

O projeto AURORA-1 demonstrou que conceitos fundamentais de computacao — estruturas de dados, logica booleana e analise estatistica simples — podem ser combinados de forma coesa para construir um sistema de decisao funcional em um cenario realista de missao espacial.

A implementacao da regressao linear do zero evidenciou que algoritmos de previsao nao exigem ferramentas complexas para ser efetivos. A inconsistencia proposital nos dados testou a robustez do sistema, que a identificou e reportou corretamente sem interromper o fluxo de execucao.

Do ponto de vista etico, o projeto reflete sobre a responsabilidade computacional em sistemas criticos: um diagnostico incorreto em uma missao espacial real poderia custar vidas. Isso reforça a importancia de codigo bem documentado, regras logicas claras e validacao de dados de entrada — principios aplicados em cada camada deste sistema.